

string

string – это неизменяемая (immutable) последовательность байт произвольной длины

```
type stringStruct struct {  
    str unsafe.Pointer // указатель на массив байт в памяти  
    len int            // длина в байтах (не в символах!)  
}
```

Ключевые аспекты:

1. “Последовательность байт” – строка НЕ является множеством символов. Это просто массив байтов, расположенных подряд в памяти.
2. “Произвольной длины” – строка может содержать любые байты, включая нулевые (в отличие от C, где строка заканчивается на \0).
3. “Неизменяемая” – после создания строки вы не можете изменить ни один её байт.

Из структуры мы можем понять:

1. Строка – срез только для чтения
2. Две разные строки могут указывать на одну и ту же область памяти

```
s := "hello"  
sub := s[1:3] // sub указывает на ту же память, что и s, но с длиной 2
```

Строки и кодировка UTF-8

В Go исходный код программы всегда в UTF-8. Литералы строк в коде тоже в UTF-8.

Но сама строка в памяти – это просто байты. Go не знает о кодировке строки. Это просто массив []byte.

Однако:

Строковые литералы по умолчанию интерпретируются как UTF-8.

Встроенная поддержка (например, range, unicode/utf8) ожидает, что строка – это валидный UTF-8, но не требует этого.

Go не накладывает ограничений: вы можете хранить в строке что угодно, даже бинарные данные.

```
s := "\x00\x01\x02" // абсолютно валидная строка с нулевыми байтами  
fmt.Println(len(s)) // 3
```

Нулевое значение строки – пустая строка (не nil!).

Проверяйте так: if s == "" или if len(s) == 0.

Неизменяемость строк

помимо прочего, благодаря иммутабельности, строки – потокобезопасны для чтения

Конкатенация

```
s := "Hello"  
s = s + " World"
```

Пошагово:

1. Создаётся новая строка в памяти (выделяется len(“Hello”) + len(“ World”) байт).
2. В эту память копируются байты из “Hello”.

3. Затем туда же копируются байты из “World”.
4. Старая строка “Hello” становится недостижимой (если на неё больше нет ссылок).
5. Сборщик мусора позже очистит старую память.

Важно:

Это не “добавление” к существующей строке, это создание новой и копирование.

Индексация строк

`s[i]` — индексация по байтам

```
s := "Привет"
b := s[0]
fmt.Printf("%d %c %T\n", b, b, b) // 208 8 uint8 (байт)
s[i] всегда возвращает byte (который uint8). Это не символ, а байт.
```

Если строка в UTF-8 и содержит многобайтовые символы, индексация `s[i]` вернёт только часть символа:

```
s := "世"
fmt.Println(s[0]) // 228 (первый байт из трёх)
```

Срез строки `s[i:j]`

```
s := "Привет"
sub := s[0:2] // "П" (первые 2 байта — одна буква)
```

Важно:

срез строки работает с байтами, а не с символами. Если вы сделаете `s[0:1]` для русской буквы, это сломает UTF-8.

Ключевая особенность:

срез НЕ копирует данные. Он создаёт новую строку, которая ссылается на ту же память.

```
s := "hello world"
sub := s[0:5]
// sub теперь указывает на ту же память, что и s, но len=5
```

Итерация по строке

```
s := "Привет"
for index, r := range s {
    fmt.Printf("байтовая позиция: %d, руна: %c, код: %U\n", index, r, r)
}
// Вывод:
// байтовая позиция: 0, руна: П, код: U+041F
// байтовая позиция: 2, руна: р, код: U+0440
// байтовая позиция: 4, руна: и, код: U+0438
```

Как это работает:

- range декодирует UTF-8 последовательности.
- Если встречается некорректный UTF-8 (байт не является валидным), range возвращает руну `U+FFFD` (символ замены) и сдвигается на 1 байт.

```
s := "При\xffвет" // невалидный байт \xff
for i, r := range s {
    fmt.Printf("%d: %U\n", i, r)
}
// 0: U+041F
```

```
// 2: U+0440  
// 4: U+0438  
// 6: U+FFFD (невалидный байт)  
// 7: U+0432  
// 9: U+0435  
// 11: U+0442
```

Руны как представления символов

rune – это псевдоним для int32. Он используется для хранения Unicode code point (номер символа в таблице Unicode).

```
type rune = int32
```

Зачем это нужно? Одна руна может занимать:

- 1 байт (ASCII: 'A', '1')
- 2 байта (кириллица: 'П', 'р')
- 3 байта (китайские иероглифы: '世')
- 4 байта (эмодзи: '🚀')

```
var r rune = '世'
fmt.Printf("%c, размер: %d байт, код: %d\n", r, unsafe.Sizeof(r), r)
// 世, размер: 4 байт, код: 19990
```

Конвертация: []byte(s) и []rune(s)

[]byte(s) – конвертация в срез байт

```
s := "hello"
b := []byte(s) // [104 101 108 108 111]
```

Создаётся копия данных. Изменение b не влияет на s.

Используйте, когда нужно работать с сырыми байтами (замена n на gn, шифрование, сериализация).

[]rune(s) – конвертация в срез рун

```
s := "Привет"
r := []rune(s) // [1055 1088 1080 1074 1077 1090]
```

Декодирует UTF-8 и создаёт копию каждого символа как int32.

Полезно, когда вам нужно обрабатывать каждый символ отдельно (переворот строки, транслитерация).

Пример: переворот строки с поддержкой Unicode

```
func Reverse(s string) string {
    runes := []rune(s) // конвертируем в руны (копия)
    for i, j := 0, len(runes)-1; i < j; i, j = i+1, j-1 {
        runes[i], runes[j] = runes[j], runes[i]
    }
    return string(runes) // конвертируем обратно в строку
}
```

Без []rune переворот сломает многобайтовые символы.

Прочие детали

Длина строки

2 понятия длины

```
import "unicode/utf8"

s := "Привет"
fmt.Println(len(s))           // 12
fmt.Println(utf8.RuneCountInString(s)) // 6
fmt.Println(len([]rune(s)))   // 6 (но медленнее, т.к. создаёт срез)
```

Эффективность конкатенации

Проблема с + в циклах

```
// Медленно (O(n²))
s := ""
for i := 0; i < 100000; i++ {
    s += "a" // на каждой итерации:
              // 1. выделяется новая память
              // 2. копируется старая строка
              // 3. добавляется "a"
}
```

Решение 1: strings.Builder

```
var b strings.Builder
b.Grow(100000) // опционально: выделяем память заранее (избегает реаллокаций)
for i := 0; i < 100000; i++ {
    b.WriteString("a")
}
result := b.String()
```

Как работает внутри:

- Builder хранит []byte как изменяемый буфер.
- При вызове WriteString он просто добавляет байты в буфер (если нужно, увеличивает его через grow).
- При вызове String() делает unsafe конвертацию буфера в строку БЕЗ копирования (использует unsafe.Pointer).
- Важный нюанс: после вызова String() не изменяйте Builder, иначе строка станет некорректной (баг Go).

Решение 2: bytes.Buffer Более старый и более “безопасный” аналог:

```
var buf bytes.Buffer
for i := 0; i < 100000; i++ {
    buf.WriteString("a")
}
result := buf.String()
```

Отличие: bytes.Buffer всегда копирует данные при String().

Решение 3: strings.Join Используйте, если у вас уже есть срез строк:

```
parts := []string{"Hello", "World", "!"}
result := strings.Join(parts, " ") // "Hello World !"
```

Сравнение строк

В Go строки сравниваются побайтово (лексикографически):

```
a := "hello"
b := "world"
fmt.Println(a == b) // false
fmt.Println(a < b)  // true (h < w)
```

Это не зависит от кодировки – сравниваются сырые байты.

Для сравнения с учётом Unicode (нечувствительность к регистру, нормализация) используйте пакет golang.org/x/text/collate.

Важное правило

Если вы работаете с текстом (символами) – используйте `range` и `[]rune`. Если с бинарными данными – используйте `[]byte`.